Tulip QA Automation Engineer Take Home

Automation and Affiliate Links Test Plan

Strategy: why Playwright with TypeScript

Pros

- Auto-waiting and web-first assertions mean fewer flaky tests
- Each test runs in a fresh, isolated browser context
- Parallel runs, fixtures, trace viewer, video and screenshots built in
- Chromium, Firefox and WebKit from one API
- TypeScript gives autocomplete and compile-time errors, free and open source

Cons

- Younger ecosystem than Selenium, fewer plugins and legacy examples
- Mobile is emulation only, WebKit is not real Safari
- Not for native desktop apps
- Also considered: Cypress (runs inside the browser, limited multi-tab), Selenium (more boilerplate, manual waits)

Architecture: specs, fixtures, page objects

How it is organised

- Specs describe what the user does, without locators
- Page objects hold every locator, so a UI change is fixed in one place
- BasePage share the cart icon and badge
- Fixture logs in as a standard_user before each test

Why

- Each test has its own browser context, so tests are independent and can run in parallel
- Credentials come from .env, which is git-ignored
- No hard coded products: tests use what the page shows
- Small enough to read it quickly, structured enough to grow

Logic: sorting test

Idea

- **Read the names or prices that are displayed, no hard-coded products, works for 6 or 1000 items**
- **Compare with a sorted copy of the same list**
- **Prices are compared as numbers, not text**
- **Start from a different order, so A to Z cannot pass just because it is default**
- **One test per sort option, so one failure does not hide the others**

Sorting code snippet

```
import { expect, test } from '../fixtures/fixtures';

const sortCases = [
  { label: 'Name (A to Z)', field: 'name', direction: 'asc' },
  { label: 'Name (Z to A)', field: 'name', direction: 'desc' },
  { label: 'Price (low to high)', field: 'price', direction: 'asc' },
  { label: 'Price (high to low)', field: 'price', direction: 'desc' },
] as const;

function sortedCopy(values: (string | number)[], direction: 'asc' | 'desc') {
  const copy = [...values].sort((a, b) =>
    typeof a === 'number' && typeof b === 'number' ? a - b : String(a).localeCompare(String(b)),
  );
  return direction === 'asc' ? copy : copy.reverse();
}

for (const { label, field, direction } of sortCases) {
  test(`sorts by ${label}`, async ({ inventoryPage }) => {
    const other = sortCases.find((c) => c.label !== label)!;
    await inventoryPage.sortBy(other.label);
    await inventoryPage.sortBy(label);

    const actual = field === 'name' ? await inventoryPage.getNames() : await inventoryPage.getPrices();

    expect(actual.length).toBeGreaterThan(1);
    expect(actual).toEqual(sortedCopy(actual, direction));
  });
}

```

Test approach: one area per acceptance criterion

Cookie persistence

- ?ref=QA_INFLUENCER sets affiliate_tracking
- Cookie expires after 30 days
- Cookie is still present after closing the browser and reopening it even without the query parameter

UI feedback

- Checkout: overview shows the Referral applied banner
- Correct influencer name for each ID
- No banner without a referral
- Long name, special characters

Attribution

- Purchase request carries the ID to the backend
- Purchase without a cookie sends no affiliate
- Several items, several quantities, then purchase
- Order is not blocked if tracking fails

Backend metrics

- Row has affiliate, amount, products, date
- Correct amounts as shown to the user
- Cancelled or abandoned checkout does not create DB entry

Edge and negative cases

- With or without cookie set, site behaves normally
- query parameter with a new id (what should happen?)
- Cookie is valid on day 29 and 30 and gone on 31
- Invalid or deactivated ID - validated on backend?
- Delete cookies - affiliate is not set

Attribution and backend checks

API

- Check the request payload for affiliate_tracking
- No cookie: field is empty or absent and the order still succeeds
- Invalid ID (what should happen)
- Test data: One id per influencer

Database

- One row per purchase, with/out affiliate ID
- Amounts and products match the order
- Correct Date in the correct format
- No row when a purchase is cancelled or failed

Questions for product

- What is this program for actually? Gets the customer a discount or will the influencer get something? If yes are these handled on the UI and backend?
- If the influencer gets commission, do we handle this? If yes when will it paid and what happens on returns and refunds?
- What should happen if the user enters a new query ID? The old one is overwritten?
- 30 days duration - is it fixed or refreshed on each visit?
- What should happen on an invalid ID? Which characters and length are valid for an ID?
- Is the consent banner required in the EU before the cookie is set?
- Is a registration needed to purchase with the affiliate ID?
- Is there any level of accessibility needed?

Beyond functional testing

- **Security:**
  - XSS in the ref value
  - Cookie is not trusted by the backend
- **Privacy:**
  - GDPR
  - Data retention and deletion
  - No personal data in cookie
- **Browser compatibility**
  - Chrome
  - Firefox
  - Safari
  - Mobile and in-app browsers
- **Reliability and Performance:**
  - Retries and idempotency
  - Tracking failure does not block checkout
- **Observability:**
  - Logs
  - Alerts
  - Traces
- **Automation and rollout**
  - API and regression tests in CI
  - Feature flags, gradual rollout, rollback